

CHAPTER 7 — CHECKING A PRIME NUMBER

SLIDE 1 — WHAT IS A PRIME NUMBER?

MEET PRIME NUMBERS!

A **Prime Number** is a number that has **exactly two factors**:

 **1 and the number itself**

Simple Definition

A Prime Number is a number that is divisible only by **1 and itself**.

LOOK AT THIS!

Take **7**:

Factors of 7:

1, 7

Only two factors →  **Prime Number**

Take **6**:

Factors of 6:

1, 2, 3, 6

More than two factors →  **Not Prime**

EASY ANALOGY

Think of a prime number as a **locked room** 

Only two keys can open it:



1



The number itself



REMEMBER!

2 FACTORS → PRIME

MORE FACTORS → NOT PRIME



SLIDE 2 — CHECKING FOR FACTORS



HOW DO WE CHECK A PRIME NUMBER?

We check whether the number can be divided **exactly** by any number between 2 and $n-1$.

For example:

Number = 7

None

$7 \% 2 \rightarrow 1$ ✗

$7 \% 3 \rightarrow 1$ ✗

$7 \% 4 \rightarrow 3$ ✗

$7 \% 5 \rightarrow 2$ ✗

$7 \% 6 \rightarrow 1$ ✗

No number divides 7 exactly.

So:



7 is Prime



KEY IDEA

We use:

None

$n \% i == 0$

If the remainder is 0, we found another factor.

★ **REMEMBER!**

% → CHECKS DIVISIBILITY

SLIDE 3 — COUNTING FACTORS

 **LET'S COUNT THE FACTORS!**

We can keep a variable called `count`.

Start with:

```
None  
count = 0;
```

Whenever we find a factor:

```
None  
count++;
```

Example: 7

```
None  
1 → factor  
7 → factor
```

So:

```
None  
count = 2
```

Therefore:

 **Prime Number**

Example: 6

None

```
1 → factor
2 → factor
3 → factor
6 → factor
```

So:

None

```
count = 4
```

Therefore:

 **Not Prime**

 **BIG MEMORY TRICK**

CHECK → COUNT → COMPARE

- `%` → CHECK
- `count++` → COUNT
- `count == 2` → COMPARE

 **TAKEAWAY**

A prime number must have **exactly 2 factors**.

SLIDE 4 — CHECKING A PRIME NUMBER

 **BASIC C CODE**

None

```
int n, i, count = 0;
```

```
scanf("%d", &n);

for (i = 1; i <= n; i++)
{
    if (n % i == 0)
    {
        count++;
    }
}

if (count == 2)
{
    printf("Prime Number");
}
else
{
    printf("Not Prime Number");
}
```

LET'S UNDERSTAND IT

For **n = 7**:

None

```
i = 1 → 7 % 1 = 0 → count = 1
i = 2 → not a factor
i = 3 → not a factor
i = 4 → not a factor
i = 5 → not a factor
i = 6 → not a factor
i = 7 → 7 % 7 = 0 → count = 2
```

Finally:

None

`count = 2`

Therefore:

✅ 7 is a Prime Number!

★ MEMORY TRICK

LOOP → CHECK → COUNT → COMPARE

SLIDE 5 — PRIME NUMBER RECAP

REMEMBER THESE!

- | Step | What we do |
|------|-------------------------------|
| 1 | Start <code>count = 0</code> |
| 2 | Loop from 1 to <code>n</code> |
| 3 | Check <code>n % i == 0</code> |
| 4 | Increase <code>count</code> |
| 5 | Check <code>count == 2</code> |

BIG MEMORY TRICK

CHECK → COUNT → COMPARE

None

`n % i == 0`

↓

FACTOR FOUND

↓
`count++`
↓
`count == 2 ?`
↓
`PRIME` ✓

Question

1 How many factors does a Prime Number have?

- A) 1
- B) 2
- C) 3
- D) 4

💡 Hint: Think about the factors of 7.

📖 Glossary:

Factor → A number that divides another number exactly.

2 What does `count` store?

None

`count++;`

- A) Number of digits
- B) Number of factors
- C) Original number
- D) Last digit

💡 Hint: We increase `count` whenever we find a factor.

 Glossary:

Count → The number of factors found.

4 Complete the Prime Check

None

```
if (count ____ 2)
{
    printf("Prime Number");
}
```



Hint: A prime number has exactly 2 factors.

 Glossary:

== → Checks whether two values are equal.

4 Find the Mistake

A student writes:

None

```
if (count == 1)
{
    printf("Prime Number");
}
```

What should be changed?

A) `count == 1` → `count == 2`

B) `count == 2` → `count == 1`

C) `count++` → `count--`

D) Nothing



Hint: Remember the number of factors a prime has.



Glossary:

Condition → A statement used to check whether something is true.

5 What happens when a factor is found?

None

```
if (n % i == 0)
{
    -----;
}
```

A) `count++`

B) `count--`

C) `count = 0`

D) `count = n`



Hint: We need to increase the number of factors found.



Glossary:

`++` → Increases a value by 1.